

CONCEPTION PHASE: Global High-Availability Website Architecture

Course: Cloud Programming (DLBSEPCP01_E)

Task: 1 - Host a Simple Webpage on AWS

Name: NDAHIRO Putine Jonathan

Matriculation Number: 4242218

ARCHITECTURE OVERVIEW

Building a cloud architecture that deploys a simple website on AWS with three critical requirements which are global low latency, automatic scaling and high availability. For this conception phase, I proposed the architecture using CloudFront CDN, Application Load Balancer, Auto scaling group, and multi-AZ EC2 deployment for achieving enterprise-grade reliability and performance at minimal cost.

1. The problem statement & Requirement Analysis

The challenge is to deploy a website that is highly available which means Users don't experience downtime when infrastructure fails, serves visitors globally without latency (users worldwide experience fast response times) and automatically scales (system handles traffic spikes without manual intervention). The reason why this is essential is because a single-server deployment usually fails on all these 3 counts; the goal is to design a system that handles these challenges seamlessly. The requirements are:

A. global Low latency

to understand latency basics helps realize how influential and impactful it is to the users. Latency is basically the time delay between a user's request and the corresponding response (time data travels). Example: If our servers are in Frankfurt and the user is in Tokyo ~ 10,000 km and data travels at light speed through fiber optic cables, i.e. $10,000 \div 300,000 \text{ km/s} = 33 \text{ milliseconds}$ data travels. to put this into perspective: <100 ms: users don't perceive delay (response is instant); 100 – 300 ms : noticeable delay, poor user experience; >300ms: users would get frustrated and leave site. The problem is in real world without routing and processing the latency is between 200 – 400 ms.

The solution is CloudFront CDN. What is a CDN(Content Delivery Network)?

CloudFront is a content delivery network that accelerates the delivery of websites , APIs and video content by copying websites to servers in 600+ cities worldwide.

How it works:

WITHOUT CloudFront:

Kigali visitor → request travels 6,000 km → Frankfurt server →
response travels 6,000 km back → 300+ ms latency which is not good

WITH CloudFront:

Kigali visitor → CloudFront Kigali edge → cached copy returns in 5 ms

(Visited before? Cache hit = instant)

(First visit? Cache miss → fetch from origin once, then cache)

CloudFront has edge locations in every major city, so when a user requests a page, CloudFront serves it from the nearest location, static content like HTML, CSS, JS, and image are cached. If content isn't cached, CloudFront fetches from your origin(ALB) once, then caches it. Cache expires after configured TTL (Time to live) after typically one hour

Why is CloudFront optimal:

APPROACH	LATENCY	COST	COMPLEXITY
CloudFront (Chosen)	5-50 ms global	Costs between \$20-50/month	Low
Multi-region EC2	100-200ms	\$200+/month	Very high
Serves from single region	300+ ms	\$10/month	Very low

REQUIREMENT 2: HIGH AVAILABILITY

Problems:

Single server failure = 100% failure

Solutions:

Multi-AZ, Application Load Balancer (ALB), Health Checks, Auto Scaling Group (ASG)

Multi-AZ: Multiple EC2 instances (2-10 servers should be running at once)

Application Load Balancer: Sends requests to only working instances; monitors every 30 seconds

Multi-AZ deployment: EC2 instances hosted in AZ 1 and AZ 2; failure in one AZ doesn't affect others

Auto scaling Group: Detects failed instances and replaces them with new instances within 60 seconds

Outcome:

99.99% availability = 4 hours failure per year (versus 8+ hours per month with single

REQUIREMENT 3: AUTO SCALING

Issue: Traffic spike caused by viral tweet (causing 10X traffic) causes servers to overload; website is not responsive.

Solution: Auto Scaling Group

CloudWatch checks the CPU usage level

CPU usage level > 70% → ASG automatically adds new instances (start-up time 60 seconds)

CPU usage level < 30% → ASG automatically removes instances

Scalability range: 2 to 10 instances depending on traffic

Advantage: No need for manual scaling; saves money during low traffic (70% less expensive than maintaining 10 servers)

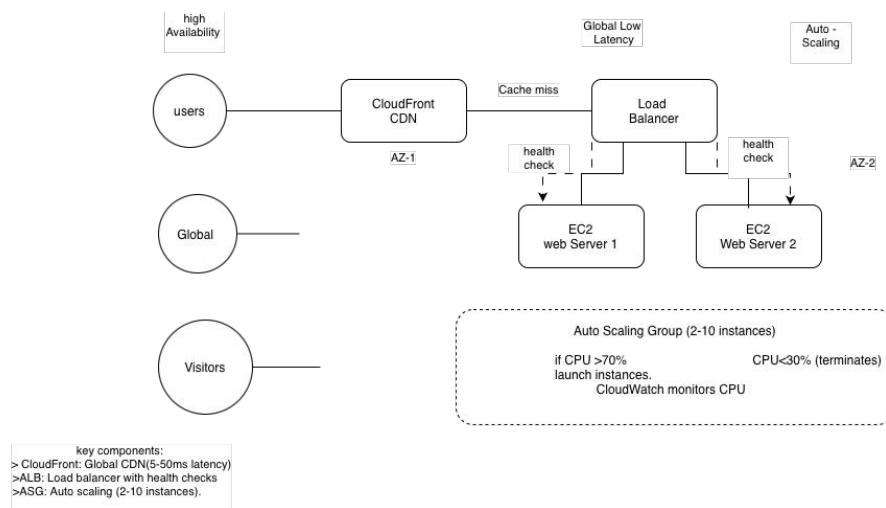
Service justification

Service (chosen)	Purpose	Why it was chosen
CloudFront	Global CDN	Solves latency; cheapest solution for global delivery
ALB	Load balancing	Distributes traffic; integrates with ASG health checks
ASG	Auto scaling	Automatic capacity management; cost efficient
Multi-AZ EC2	Virtual servers	Redundancy across zones; high availability

VPC	Network security	Isolation; security best practice
-----	------------------	-----------------------------------

Cost estimate:

component	Cost per month
CloudFront	\$ 20 - 50
ALB	\$ 16
EC2	\$ 22.50
Total	\$ 60 – 85/month



Conclusion:

This architecture will solve all three requirements: CloudFront eliminates global latency, Multi- AZ + ALB +ASG providing high availability and automatic failover, and Auto Scaling Group managing capacity automatically. Cost-efficient (about 85\$/month), scalable, and reliable